

Lab 2a

Breadth-First-Search (BFS)

Imagine a group of children entering a large school to find a prize hidden in one of the classrooms.

The classrooms are named as A, B, C, D, E, F, G and H. The children start from classroom A, and the prize is hidden in classroom H. They know which classrooms are connected to each other, but they do not know the best way to reach the prize. Instead of running around randomly, they search in a careful and organized way.

The children first stand in classroom A. From there, they can see that there are two classrooms they can go to next, B and C. Since both classrooms are equally close to where they are standing, they decided to visit both of them before moving any farther. They do not ignore one classroom to rush into another. Their idea is very simple: first check everything that is nearby, and only then move farther away.

After visiting B and C, they continue their search. From classroom B, they find classrooms D and E. From classroom C, they find classrooms F and G. They continue checking these classrooms one by one because they are all at the same distance from the starting point. Only after they have finished checking all these nearby classrooms do they move to the next possible place.

while exploring classroom E, they notice another classroom connected to it, which is H. As soon as they enter H, they find the prize. The path they followed to reach the prize is $A \rightarrow B \rightarrow E \rightarrow H$. Although there were many classrooms in the building, they reached the destination without missing any nearby classroom. This method makes sure that the shortest possible route is found because the search always moves outward step by step inside of rushing deep into one direction.

Think of this like searching for a friend in a shopping mall. You would first check all the shops on the current floor before taking the escalator to another floor. It would not make sense to go to the top floor immediately while ignoring the nearby shops. This is exactly how this searching method works. It carefully checks everything close by before moving farther away, making the search organized and efficient.

Now let us see how this idea is written in the program. The classrooms and their connections are stored in a graph. The search begins from class A, and the goal is to reach classroom H. The program keeps a list that stores the possible paths that are waiting to be explored. At the beginning, this list contains only one path, which starts with A.

During every step, the program always removes the first path from the waiting list using the statement:


```
path = queue.pop(0)
```

This follows the First In, First Out (FIFO) rule. The path that has been waiting for the longest time is explored first. Because of this rule, all the nearby classrooms are explored before moving to classrooms that are farther away.

The program also keeps another list called visited. Everytime a classroom is explored, its name is added to this list. This prevents the same classroom from being explored repeatedly. Whenever the program finds another connected classroom, it creates a new path by adding that classroom to the current path and places it at the end of the waiting list. This process continues until classroom H is reached.

For this example, the program returns the path:

```
['A', 'B', 'E', 'F']
```

This is the shortest path from A to H. It also returns the orders in which the classrooms were explored.

Breadth-First Search and Depth-First search are almost identical, but they differ by only one important line. In Breadth-First search, the program uses:

```
path = queue.pop(0)
```


which removes the first path from the waiting list. In Depth-First search, it uses:

`path = stack.pop()`

which removes the last path instead. This single change completely changes the searching method. BFS explores every nearby place first and then moves farther away, while DFS continues along one path as far as possible before coming back to explore another path.